



LLM 활용 소프트웨어

1. 시스템 개요

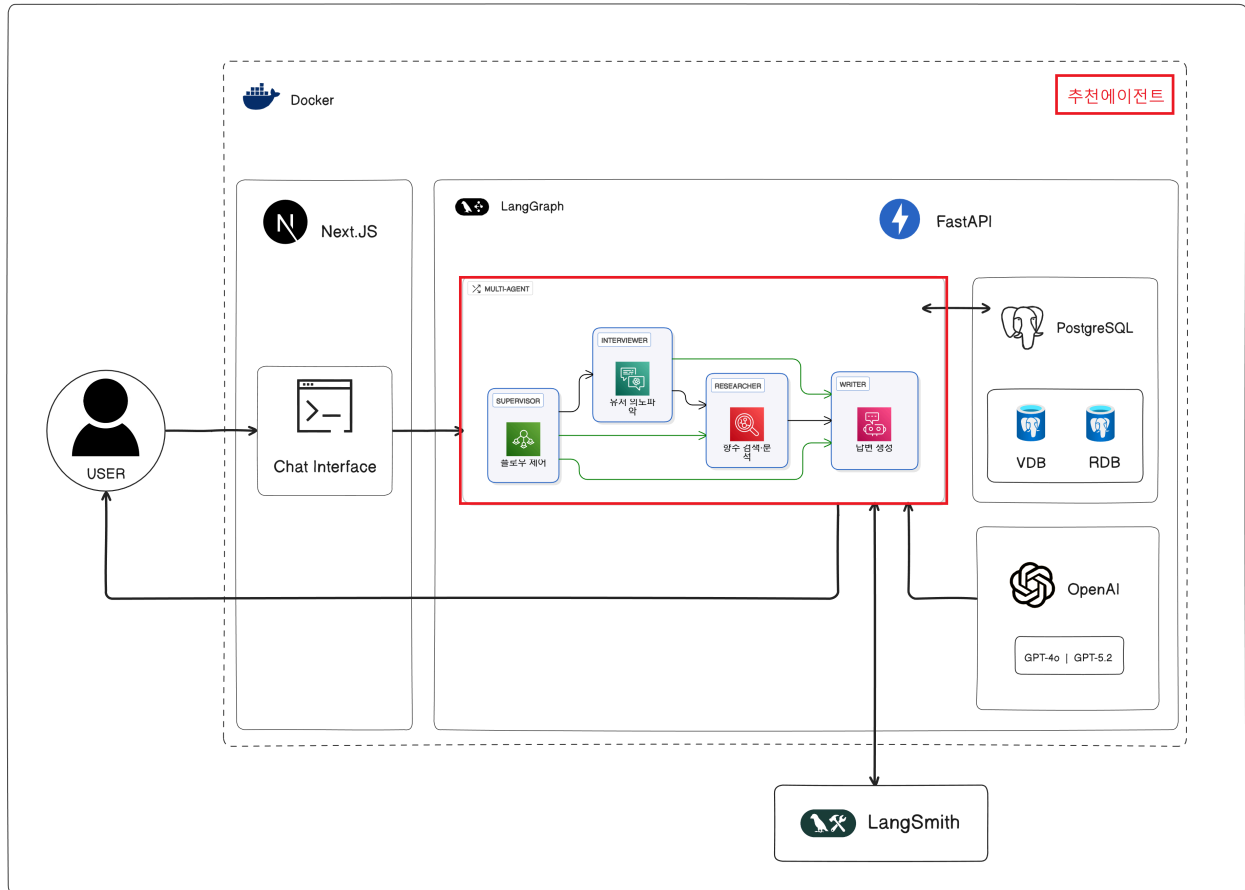
본 시스템은 대화를 통해 요구사항을 분석하여 사용자 정보에 최적화된 향수를 제안하며 회원 사용자에게는 개인화된 향수 관리 기능을 함께 제공하는 통합 향수 추천 및 관리 서비스이다.

전체 시스템은 사용자 인터페이스를 담당하는 프론트엔드, 비즈니스 로직과 API 처리를 담당하는 백엔드, AI 기반 추천 로직을 수행하는 LangGraph 기반 파이프라인, 그리고 관계형 데이터와 벡터 데이터를 통합 관리할 수 있는 데이터베이스로 구성된다.

추천 과정은 LangGraph 기반 멀티 에이전트 구조로 설계되었으며, 각 에이전트는 명확히 분리된 역할을 수행하고 사용자와의 대화 상태에 따라 유연하게 흐름이 제어되도록 구성되었다.

2. 전체 시스템 구성도

■ 시스템 구성도



■ 프로젝트 구조

- 코드 링크
 - [backend/graph.py](#)
 - [backend/tools.py](#)

```

Scentence/
├── backend/
│   ├── database.py    # DB 연결 관련 코드
│   ├── schemas.py    # 데이터 구조 정의
│   ├── tools.py      # 도구 모음
│   ├── graph.py      # 에이전트(Supervisor/Interviewer/Researcher/Writer)
│   ├── main.py       # 서버 실행 파일
│   ├── requirements.py
│   └── Dockerfile
└── .env

```

```

├─ prondend/      # 프론트 엔드
├─ postgres/     # DB 데이터 폴더
├─ init-db.sh     # DB 테이블 생성 스크립트
├─ init-data.sh   # 초기 데이터 적재 스크립트
└─ docker-compose.yml # 전체 서비스 실행

```

3. 추천 시스템

■ 함수 추천 프로세스

• State 정의

상태변수	역할	타입	비고
user_query	사용자 입력 쿼리	str	공통
messages	대화 히스토리 (Supervisor 판단용)	List[Any]	공통
route	라우팅	Literal[]	공통
final_response	최종 응답	str	공통
interview_context	누적된 사용자 취향 정보	str	interviewer
active_mode	interviewer 제어용	Optional[str]	interviewer
search_plans	Researcher가 수립한 전략 리스트	List[dict]	researcher
search_logs	실행 로그 (프론트엔드 전송용)	List[dict]	researcher
research_result	최종 검색 결과 텍스트	str	researcher
retry_count	Researcher 재시도 횟수	int	researcher

■ 에이전트(모듈)별 프로세스

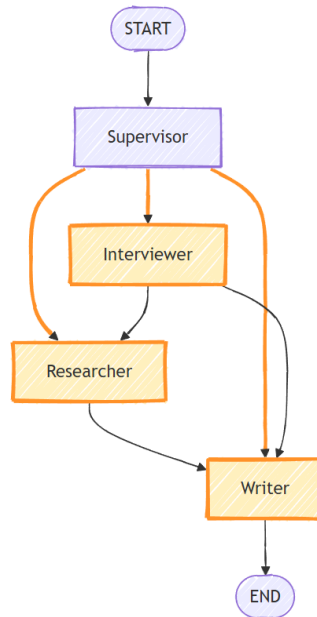
※ 프로젝트 중간점검 기준 작성된 내용, 개선 과정에서 추가 및 변경될 수 있음

(1) Supervisor

역할

Supervisor는 사용자 응답으로부터 현재 대화 상태에서 수행해야 할 다음 에이전트를 결정하고, 대화 흐름을 LangGraph 상에서 분기 및 제어하는 라우팅 전용 에이전트

에이전트 내부 행동 흐름도



절차

- (1) 입력된 사용자 질의를 기준으로 어떤 유형의 대화인지, 향수 추천에 정보가 충분한지 판단
- (2) 판단 결과에 따라 다음 에이전트로의 라우팅 경로(route)를 결정

output:
{ "route": "interviewer" | "researcher" | "writer" }

* 상세 행동 지침 프롬프트 깃링크

예외 처리

active_mode State값이 interviewer인 경우

- 판단 생략하고 interviewer로 라우팅


```
output:
{ "route": "interviewer" }
```

내부 오류 발생 시

- 판단 실패로 간주하여 안전 경로인 writer로 라우팅하여 사용자 응답을 종료 처리

```
output:
{ "route": "writer" }
```

코드

사용자 질의 분석 및 대화 유형 판단하여 route 결정

```
...
msg = client.chat.completions.create(
    model=FAST_MODEL,
    messages=[{"role": "user", "content": prompt}],
    response_format={"type": "json_object"},
)

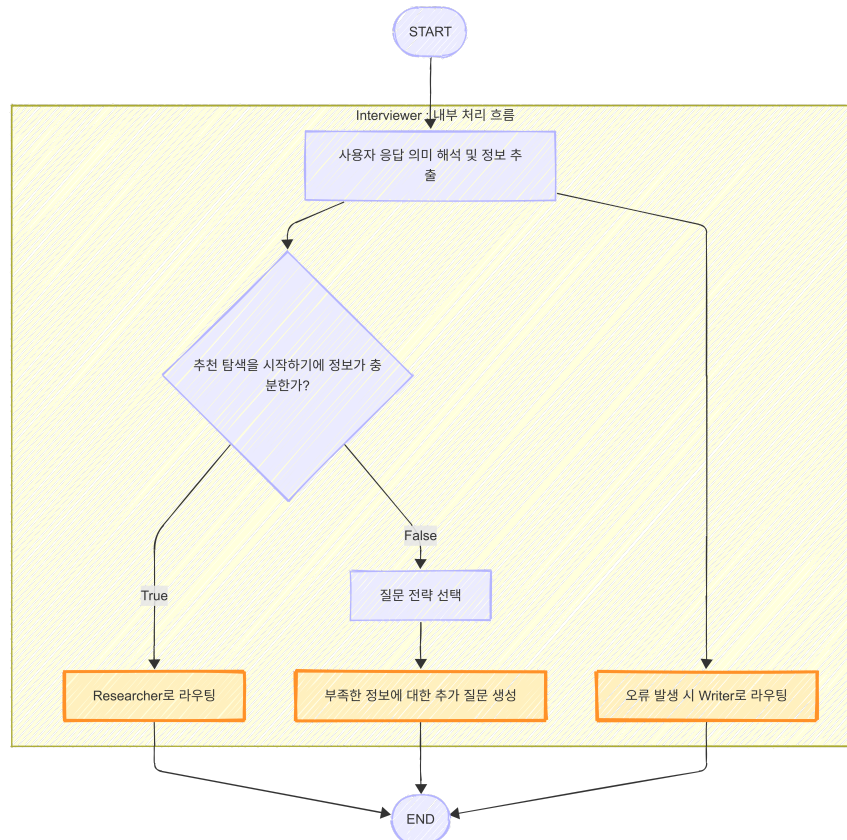
route = safe_json_parse(msg.choices[0].message.content).get("route", "writer")
...
```

(2) Interviewer

역할

Interviewer는 사용자 응답으로부터 향수 추천에 필요한 정보를 추출하고, 현재 수집된 정보가 추천 탐색을 시작하기에 충분한지 판단하여 추가 질문 또는 다음 단계로의 진행을 결정하는 에이전트

에이전트 내부 행동 흐름도



절차

(1) 사용자 응답으로부터 향수 추천에 필요한 정보를 추출하고 구조화

- 이미지(외부에서 인식되는 분위기/어울림)
- 취향(사용자 또는 대상이 선호하는 향의 느낌)
- 대상 정보(주로 성별, 필요 시 연령)
- 브랜드 정보(보조 판단 요소, 필수 아님)

(2) 추출된 정보의 조합이 추천 탐색을 시작하기에 충분한지 판단

- 위 핵심 정보가 불충분한 경우, 결손된 정보 유형에 따라 질문 전략을 선택하여 추가 정보를 유도하는 질문을 반환

```
# "is_sufficient" is false
output:
{
  "route": "end",
  "interview_context": updated_context,
  "final_response": judge_result.get("next_question"),
  "active_mode": "interviewer"
}
```

- 추천 판단이 가능한 수준으로 정보가 확보되면 Researcher 단계로 라우팅

```
# "is_sufficient" is ture
output:
{
  "route": "researcher",
  "interview_context": updated_context,
  "user_query": f"{updated_context} (사용자 의도 반영)",
  "active_mode": None
}
```

* 상세 행동 지침 프롬프트 링크

예외 처리

내부 오류 발생 시

- 인터뷰 내 동작을 중단하고 Writer로 라우팅
- 재요청을 유도하는 안전 응답을 반환

```
output:
{
  "route": "writer",
  "final_response": "잠시 문제가 생겼습니다. 다시 말씀해 주시겠어요?",
  "active_mode": None,
}
```

코드

Interviewer는 정보 수집 단계에서 “추천 탐색이 가능한 상태인가?”를 LLM이 판단하고, 해당 결과를 is_sufficient로 저장하여 분기 처리

```
judge_result = safe_json_parse(judge_msg.choices[0].message.content)

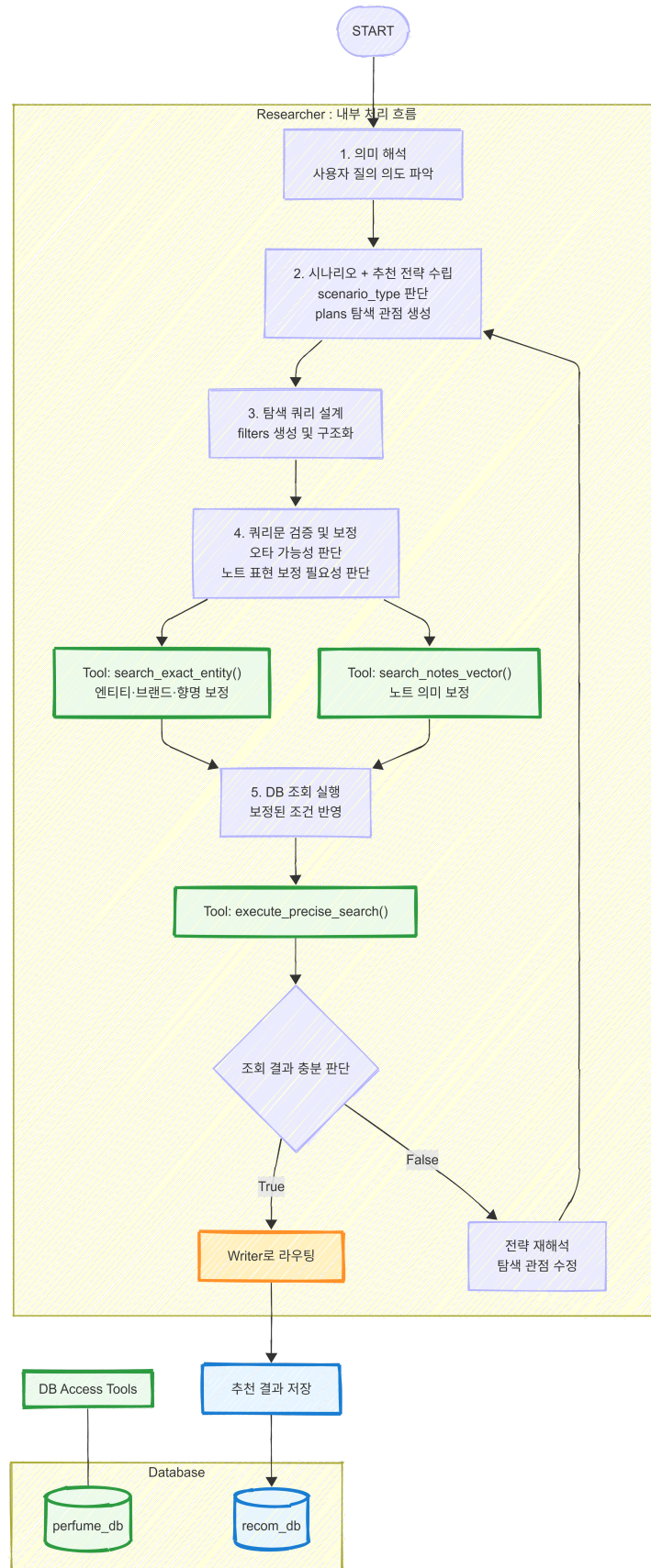
if judge_result.get("is_sufficient"):
    return {
        "route": "researcher",
        ...
        "active_mode": None,
    }
else:
    return {
        "route": "end",
        "final_response": judge_result.get("next_question"),
        "active_mode": "interviewer",
    }
```

(3) Researcher

역할

Researcher는 사용자 질의를 분석하여 시나리오에 따른 탐색 전략을 결정하고, DB 조회 조건을 생성하여 검색을 수행하여 Writer가 사용할 `research_result`를 생성하는 추천 함수 탐색 및 선별 전용 에이전트

에이전트 내부 행동 흐름도



절차

- (1) 사용자 질의를 해석하여 함수 탐색에 필요한 핵심 정보를 수집
- (2) 해석된 의도에 따라 적절한 탐색 시나리오와 추천 전략을 수립
- (3) 전략에 기반해 질의를 보정하여 DB를 조회
- (4) 유효한 추천 결과를 research_result 에 저장하여 Writer로 라우팅

```
output:
{
  "search_plans": plans,
  "search_logs": search_logs,
  "research_result": research_result,
  "route": "writer",
  "retry_count": retries + 1
}
```

* 상세 행동 지침 프롬프트 킷링크

예외 처리

내부 오류 발생 시

- 판단 실패로 간주하여 안전 경로인 writer로 라우팅하여 사용자 응답을 종료 처리

```
output:
{
  "research_result": "오류 발생",
  "route": "writer"
}
```

코드

- 자연어를 DB가 이해할 수 있는 조건으로 해석(사용자 언어 → DB 허용 값 매핑)

```
# DB 허용 값 로딩
valid_seasons, valid_occasions, valid_accords = fetch_meta_filters()

# 프롬프트 내 허용 값 기반 매핑 지침
=== [1. 보유 데이터 매핑 (Data Mapping)] ===
```

3. ****Season (계절)****:

- ****[허용된 값]****: [{valid_seasons}]
- 사용자가 "여름"이라고 하면
'Summer'를 찾아 filters에 설정

...

- 의미 기반 하이브리드 탐색
 - DB 허용 값이 있으면 → filters
 - DB에 없는 표현 처리 → note_keywords + vector

시나리오별 전략 분기 지침

****Type A. [이미지/분위기]****

- DB 허용 값에 없는 추상적 표현은
'note_keywords'에 넣어 벡터 검색

LLM이 생성한 전략 구조(JSON 예시 일부)

```
{
  "strategy_name": "포근한 비누 향",
  "filters": [],
  "note_keywords": ["Soap", "Clean"],
  "use_vector_search": true
}
```

- 질의 성격에 따른 조건 생성 방식
 - 시나리오 타입, 전략(Plan)은 LLM이 선택하고 생성

```
msg = client.chat.completions.create(
    model=HIGH_PERFORMANCE_MODEL,
    messages=[{"role": "user", "content": prompt}],
    response_format={"type": "json_object"},
)
```

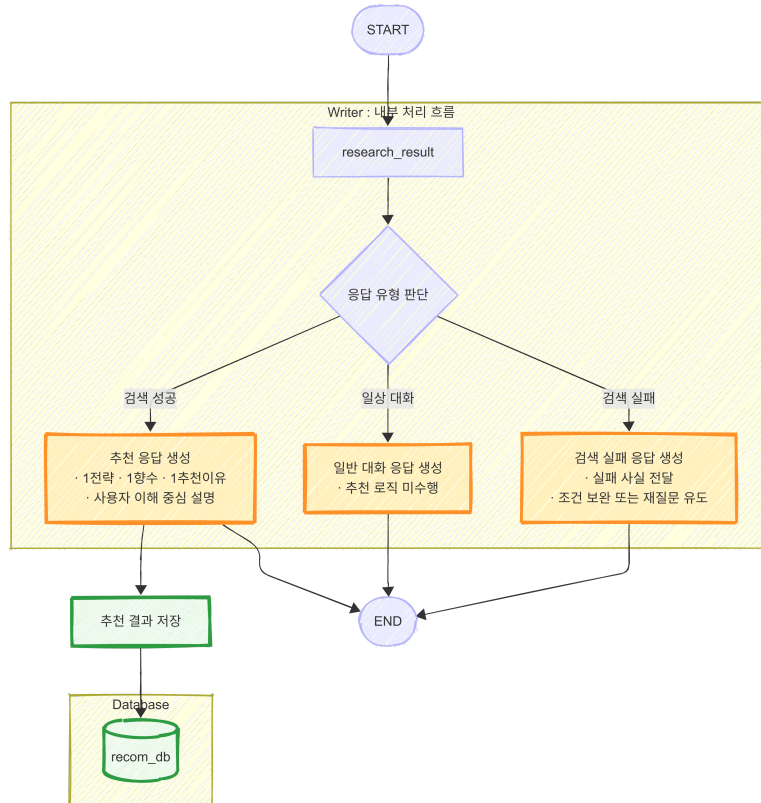
```
parsed = safe_json_parse(msg.choices[0].message.content)
plans = parsed.get("plans", [])
scenario_type = parsed.get("scenario_type", "Unknown")
```


(4) Writer

역할

Writer는 research_result 상태값을 기준으로 응답 유형을 판별하고, 응답 유형 케이스별 어떤 형식으로 답변을 출력할지 책임지는 답변 생성 및 출력 전용 에이전트

에이전트 내부 행동 흐름도



절차

- (1) research_result 상태값을 기준으로 응답 유형을 판별
- (2) 판별된 응답 유형에 따라, 사전에 정의된 형식의 답변 생성 절차를 선택하여 수행
- (3) 응답 유형별 규칙에 맞춰 사용자에게 전달할 최종 답변을 생성하고 출력

```
output:
{
  "final_response": fixed_content,
  "active_mode": None
}
```

* 상세 행동 지침 프롬프트 킷링크

예외 처리

내부 오류 발생 시

- 사용자에게 시스템 오류를 알리는 안전 메시지를 반환

```
output:
{
  "final_response": "죄송합니다. 답변 생성 중 시스템 오류가 발생했습니다.",
  "active_mode": None
}
```

코드

- Researcher 결과 상태값에 따른 지시 프롬프트 분기

```
result = state.get("research_result", "")

if "검색 결과가 없습니다" in result or "오류 발생" in result:
    system_instruction = f""" ... """
elif not result.strip():
    system_instruction = f""" ... """
else:
    system_instruction = f""" ... """
```